IF 260
Sistem Operasi

Minggu 14

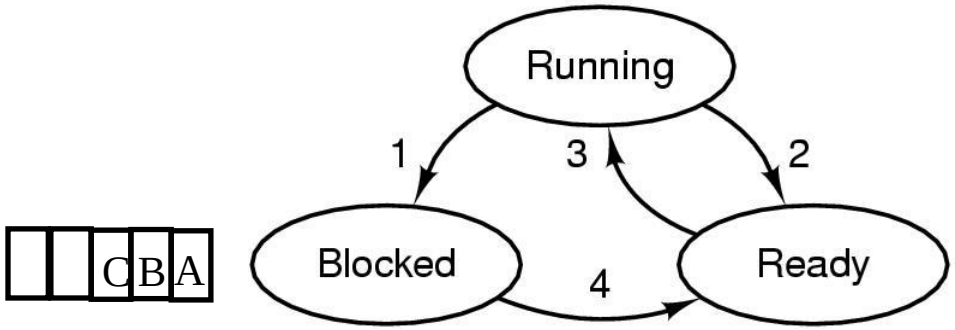
Dr. Ananda Kusuma
e-mail: ananda.kusuma@lecturer.umn.ac.id

Universitas Multimedia Nusantara
Serpong, Tangerang

Agenda

- **Review:**
  - **Deadlock detection**
  - **Paging**
  - **Page replacement algorithms**
  - **Disk scheduling algorithms**

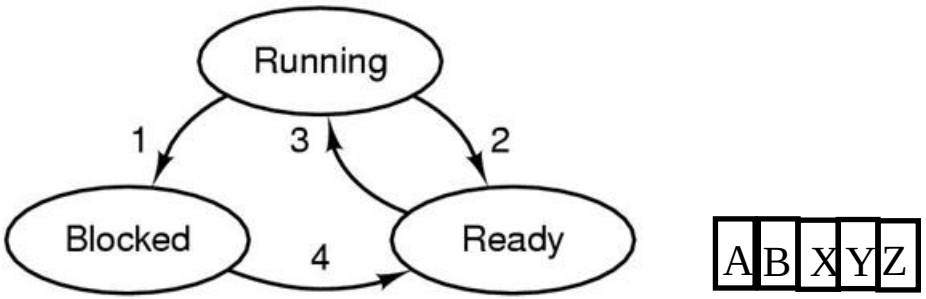
Deadlock



1. Process blocks for input
2. Scheduler picks another process
3. Scheduler picks this process
4. Input becomes available

- **Diberikan suatu himpunan yang berisikan beberapa process. Dikatakan deadlock apabila tiap process di dalam himpunan ini menunggu event yang hanya bisa dimunculkan oleh process lain di himpunan. Contoh:**
  - **Producer-consumer dgn race condition, producer dan consumer mencapai kondisi sleep dan saling menunggu untuk di-wakeup**
  - **Device deadlock: misal sistem punya 2 tape drives. Ada dua process yang gunakan masing-masing tape drive dan saling menunggu tape drive yang lain**
  - **Apabila hanya ada satu process di himpunan, maka deadlock dapat terjadi pada level thread, dan menjadi tanggung jawab programmer untuk menangani**

Starvation dan Livelock

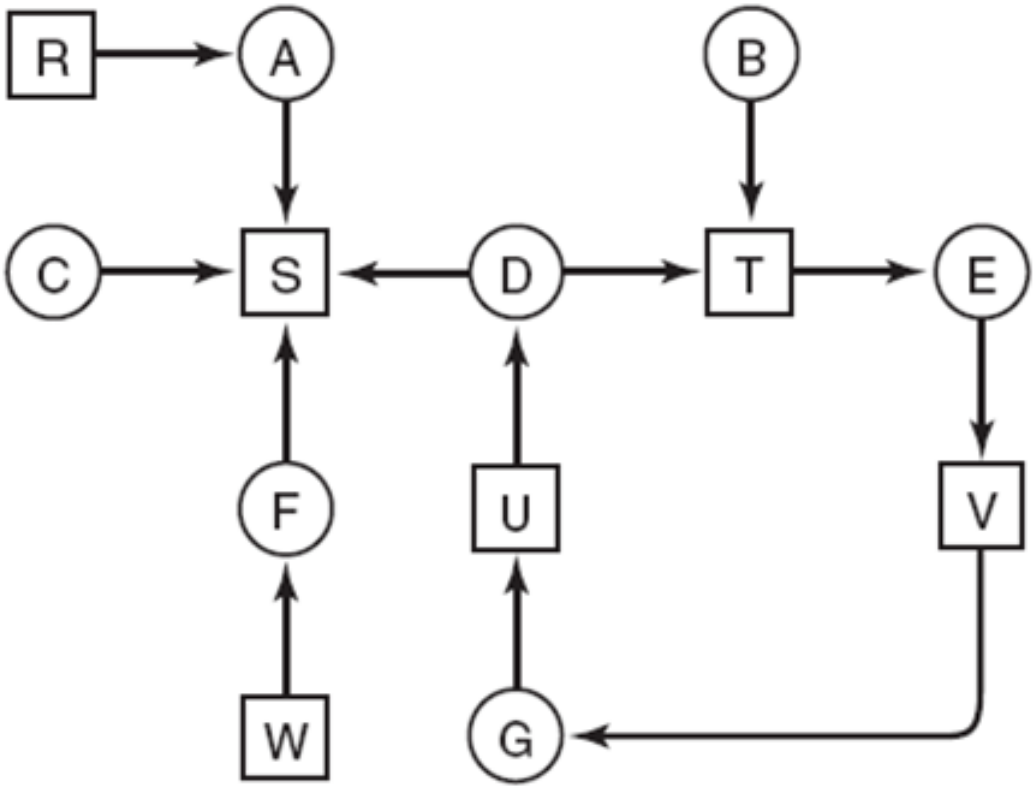


- **Starvation: Suatu kondisi di mana process berada di state ready (tidak dalam kondisi blocked), akan tetapi tidak pernah mendapatkan jatah CPU. Ini dapat terjadi karena diberikan prioritas rendah, dan CPU sibuk melayani process-process lain dengan prioritas lebih tinggi.**

- **Livelock: suatu kondisi di mana walau process dapat menggunakan CPU (state running), namun tidak dapat melanjutkan eksekusi sampai selesai. Misalnya apabila sinkronisasi antar process menggunakan busy waiting (spinlock), dan dalam kondisi tiap process saling menunggu lock dilepaskan untuk masuk ke critical region**

Contoh Penggunaan Graph

- **Deteksi deadlock dengan satu resource untuk tiap jenis.**
- **Contoh di bawah: A holds R, want S, B holds nothing, wants T, dst**
- **Apakah ini deadlock?**



Deadlock Detection: Multiple resources of each type

- **Notasi Matrix dan Vector**
  - **E: existing resource vector**
  - **A: Available resource vector**
  - **C: Current allocation matrix**
  - **R: Request matrix**

Persyaratan:

$$\sum_{i=1}^{n} C_{ij} + A_j = E_j$$

Resources in existence
$(E_1, E_2, E_3, \ldots, E_m)$

Resources available
$(A_1, A_2, A_3, \ldots, A_m)$

Current allocation matrix

$$\begin{bmatrix} C_{11} & C_{12} & C_{13} & \cdots & C_{1m} \\ C_{21} & C_{22} & C_{23} & \cdots & C_{2m} \\ \vdots & \vdots & \vdots & & \vdots \\ C_{n1} & C_{n2} & C_{n3} & \cdots & C_{nm} \end{bmatrix}$$

Row n is current allocation to process n

Request matrix

$$\begin{bmatrix} R_{11} & R_{12} & R_{13} & \cdots & R_{1m} \\ R_{21} & R_{22} & R_{23} & \cdots & R_{2m} \\ \vdots & \vdots & \vdots & & \vdots \\ R_{n1} & R_{n2} & R_{n3} & \cdots & R_{nm} \end{bmatrix}$$

Row 2 is what process 2 needs

Deadlock Detection: Multiple resources of each type

- Untuk tiap process P_i, di mana baris ke i dari matrix R kurang atau sama dengan vector A → artinya process P_i masih bisa berjalan
- Kalau ada, *mark* (tandai) process P_i, update baris ke i dari C dengan menambahkannya dengan baris ke i dari R, dan update vector A
- Kemudian tambahkan baris ke i dari C ke vector A (artinya process P_i release semua resources setelah selesai)
- Lanjutkan untuk semua process. Apabila ada process yang tidak ditandai (*unmarked*), maka terjadi deadlock
- Contoh:

Tape drives Plotters Scanners CD Roms

E = (4 2 3 1)

Tape drives Plotters Scanners CD Roms

A = (2 1 0 0)

Current allocation matrix

$$C = \begin{bmatrix} 0 & 0 & 1 & 0 \\ 2 & 0 & 0 & 1 \\ 0 & 1 & 2 & 0 \end{bmatrix}$$

Request matrix

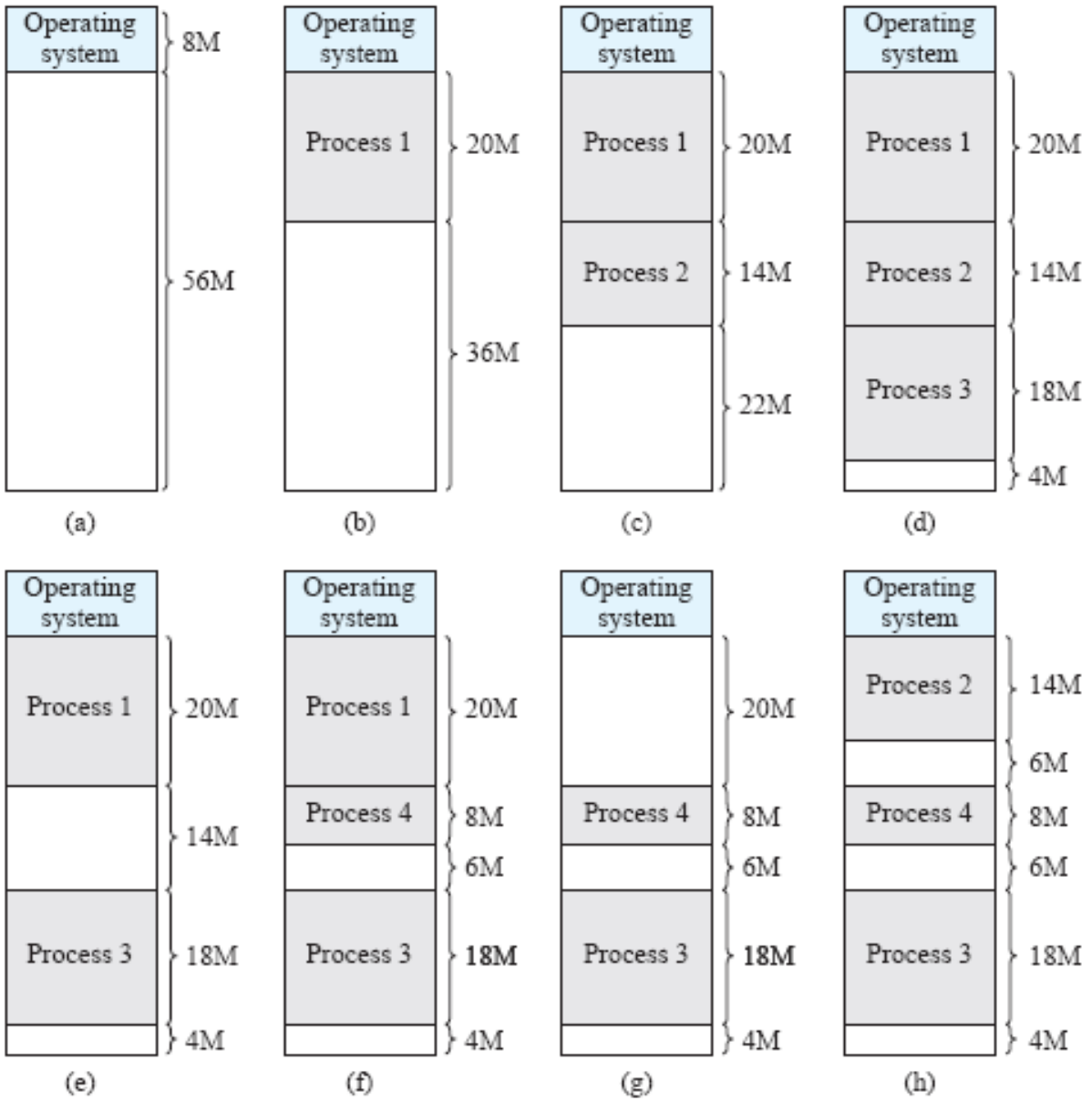
$$R = \begin{bmatrix} 2 & 0 & 0 & 1 \\ 1 & 0 & 1 & 0 \\ 2 & 1 & 0 & 0 \end{bmatrix}$$

Memory Management:
- Dynamic Partition
- Konsep Paging

Dynamic Partition

- **Partisi memory jumlah dan ukuran yang dinamis**
- **Saat process dibawa ke memory, alokasikan partisi memory sesuai dengan ukuran process**
- **Problem yang dapat terjadi: external fragmentation (bagian memory di luar partisi menjadi semakin fragmented)**
- **Untuk mengatasi external fragmentation**
  - **Compaction**
    - **OS dari waktu ke waktu memindahkan process-process di memory sehingga mereka bersebelahan dan free memory menjadi satu blok.**
  - **Placement algorithm (algoritma untuk meletakkan process pada blok yang sesuai) → lihat di slide berikutnya**

Ilustrasi Dynamic Partition



Dynamic Partition: Placement Algorithm

- **Best-fit: memilih blok dengan ukuran yang paling mendekati permintaan**
  - **Pros: mengurangi blok memori yang terbuang**
  - **Cons: search keseluruhan blok memory yang kosong**
- **First-fit: scan memory dari awal dan pilih blok pertama yang ukurannya mencukupi permintaan**
  - **Pros: Cepat**
  - **Cons: Menyisakan blok-blok berukuran kecil di awal**
- **Next-fit: scan dari posisi peletakkan terakhir dan pilih blok pertama yang ukurannya mencukupi permintaan. Kinerja serupa dengan First-Fit, tapi perlu tambahan pointer untuk mengetahui posisi terakhir**
- **Worst-fit: memilih blok dengan ukuran yang paling besar dibandingkan dengan permintaan**
  - **Pros: mengurangi blok memory kosong yang berukuran kecil**
  - **Cons: search keseluruhan blok memory yang kosong, mengurangi blok memory kosong yang mungkin diperlukan oleh process yang berukuran besar**

Contoh

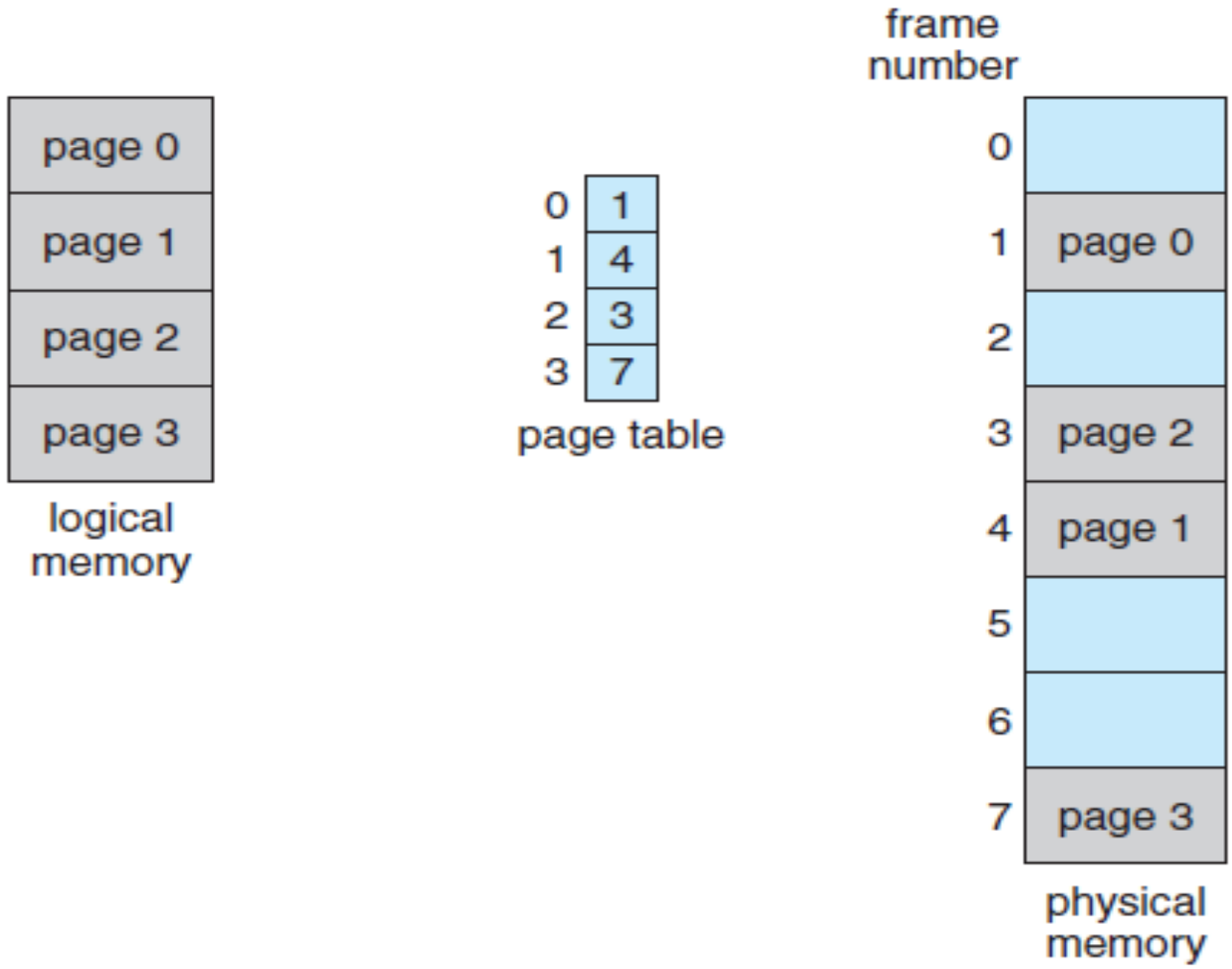
- **Pada sistem komputer yang menggunakan dynamic partition dan swapping, diketahui pada memori utama nya terdapat blok-blok memori kosong (holes) dengan urutan dan ukuran sbb: 12Kb, 7Kb, 20Kb, 18Kb, 7Kb, 9Kb, 12Kb, dan 10Kb**

- **Blok-blok mana yang akan diberikan jika ada permintaan memori berturut-turut 9KB, 10KB dan 12KB, dengan algoritma sebagai berikut?**
  - **Best Fit**
  - **First Fit**
  - **Next Fit**
  - **Worst-Fit**

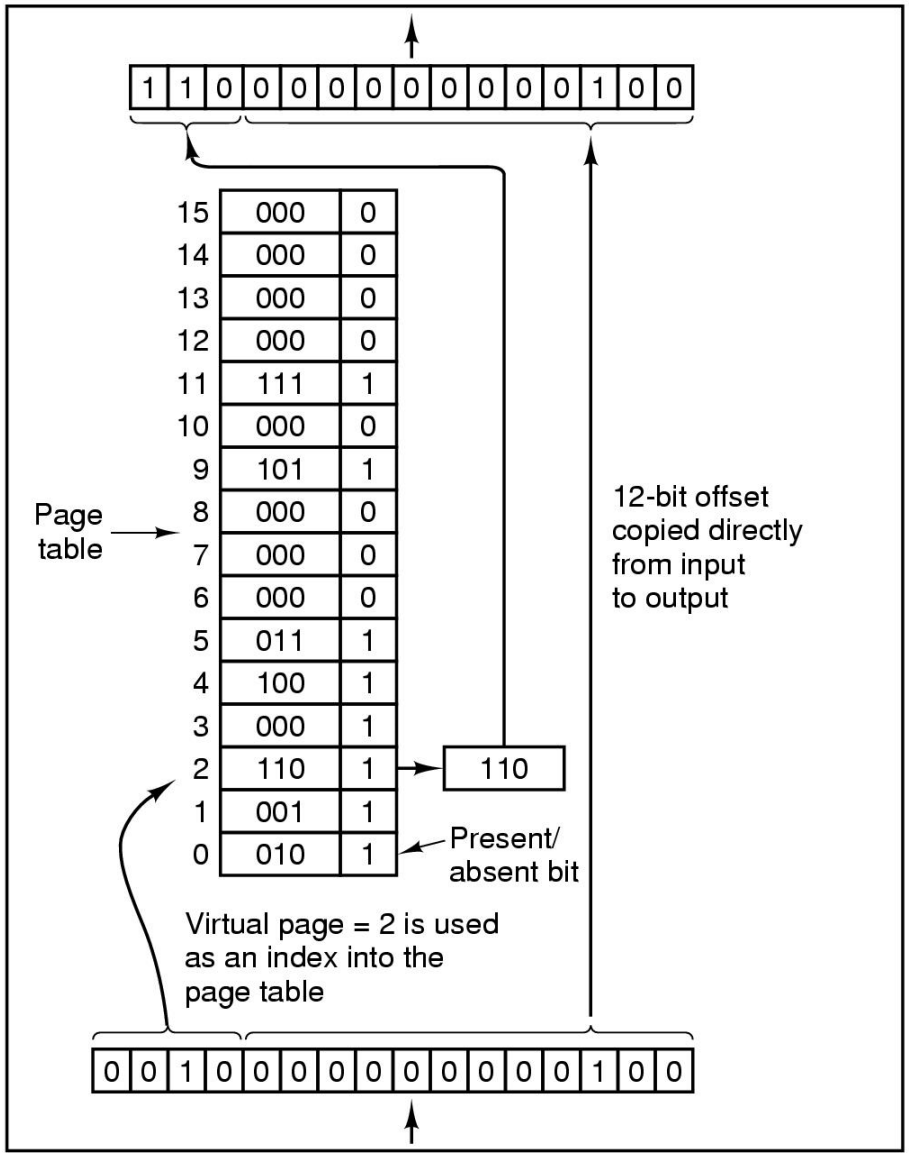
Paging

- **Bagi virtual address space atas unit-unit kecil yang berukuran tetap yang disebut dengan Page**
- **Bagi physical address space atas unit-unit kecil yang disebut Page Frame, dan ukurannya sama dengan Page**
- **MMU melakukan pemetaan Page ke Frame, dan Frame tidak harus contiguous (berurutan) pada physical memory**
- **Penggunaan Page ukuran kecil dan tetap menghilangkan external fragmentation dan mengurangi internal fragmentation (umumnya 1/2 page untuk tiap process)**

Paging: pemetaan



Contoh generik operasi internal pada MMU untuk 16 buah 4-KB Pages

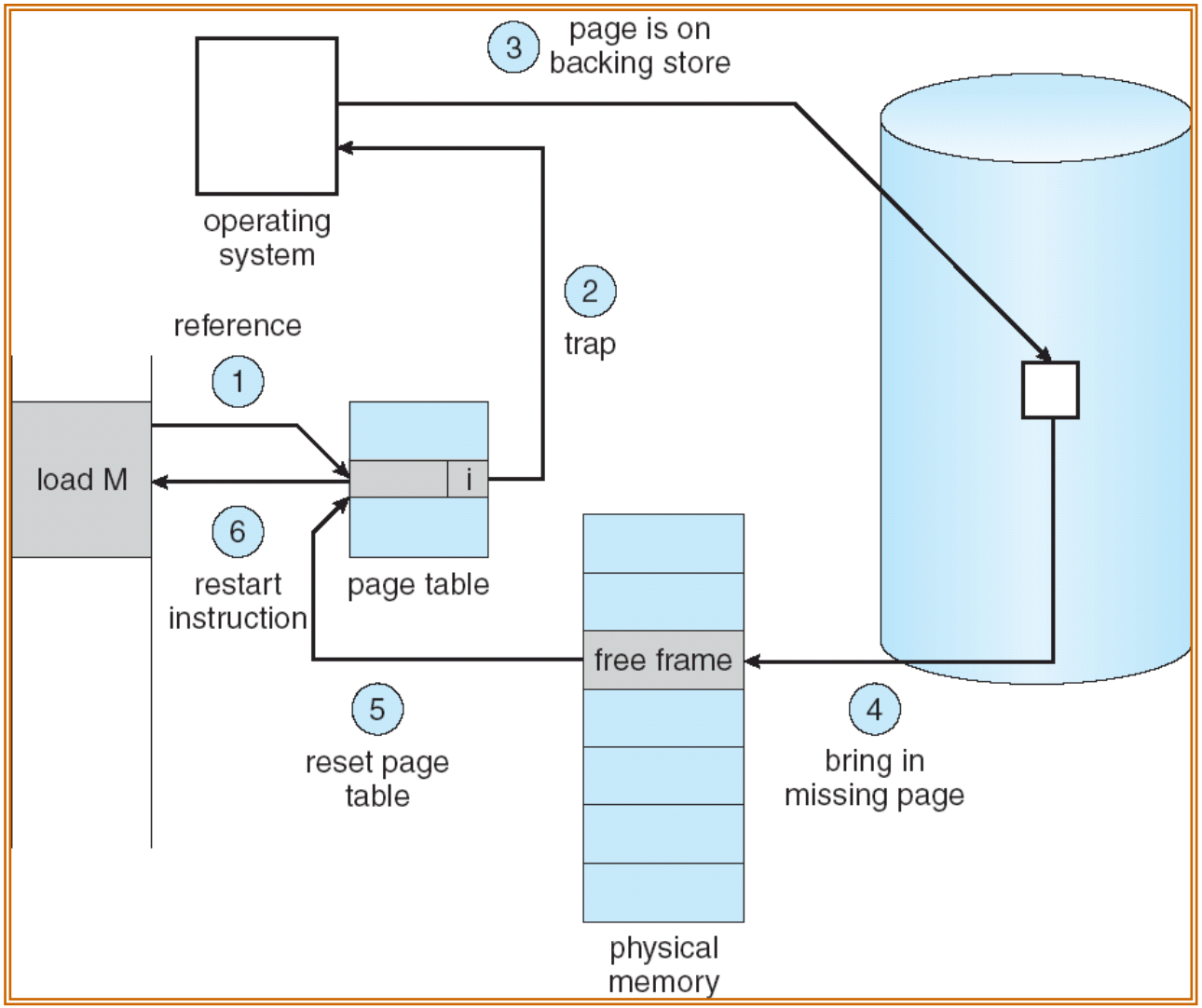


Physical address:
Page frame number & offset

Virtual page number merujuk pada Index dari page table
Page table dikelola oleh memory manager (kernel)
Page table entry berisikan page frame number

Virtual address:
Virtual page number & offset

Apabila terjadi Page Fault



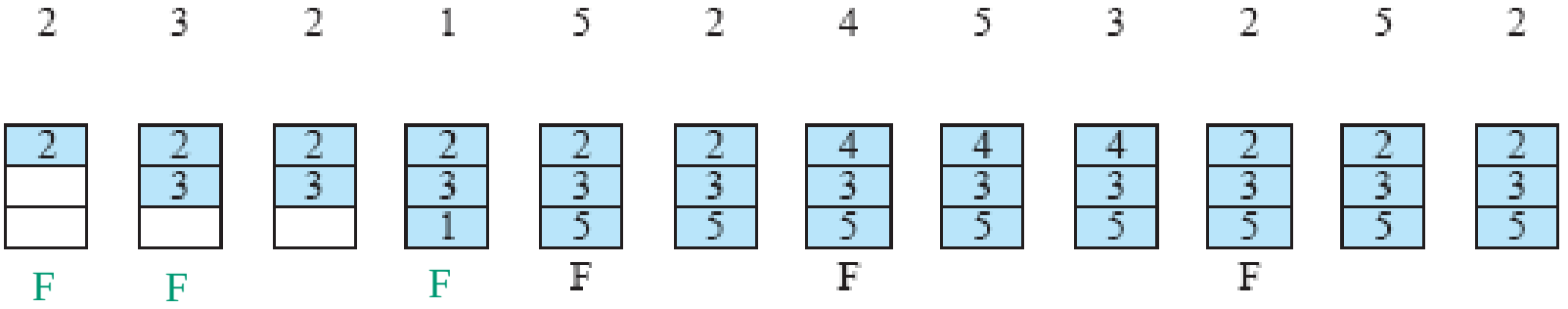
Page Replacement Algorithms

Page Replacement Algorithm

- **Jika ada page-fault untuk penggantian page, OS harus memilih page untuk dikeluarkan dari memory, sehingga ada tempat untuk page yang baru diminta**
  - **Jika page yang dikeluarkan berstatus modified (dirty bit = 1), maka harus ditulis ulang ke disk**
  - **Page yang dikeluarkan dari memory dapat merupakan page dari process yang menyebabkan page-fault, ataupun page dari process lain**
- **Kinerja sistem bagus apabila page-fault rendah**
  - **Pilih page replacement yang menghasilkan page-fault terendah**
  - **Trade-off optimality vs complexity**
  - **Evaluasi dengan menjalankan string-string tertentu dari memory reference,dan hitung jumlah page fault.**

Optimal Page Replacement Algorithm (Belady's Algorithm)

- **Memilih page yang nantinya akan diacu paling terakhir untuk dikeluarkan**
  - **Optimal (page fault paling rendah), tetapi tidak realistik karena mensyaratkan OS punya informasi sempurna akan event-event yang akan terjadi di depan**
- **Contoh:**



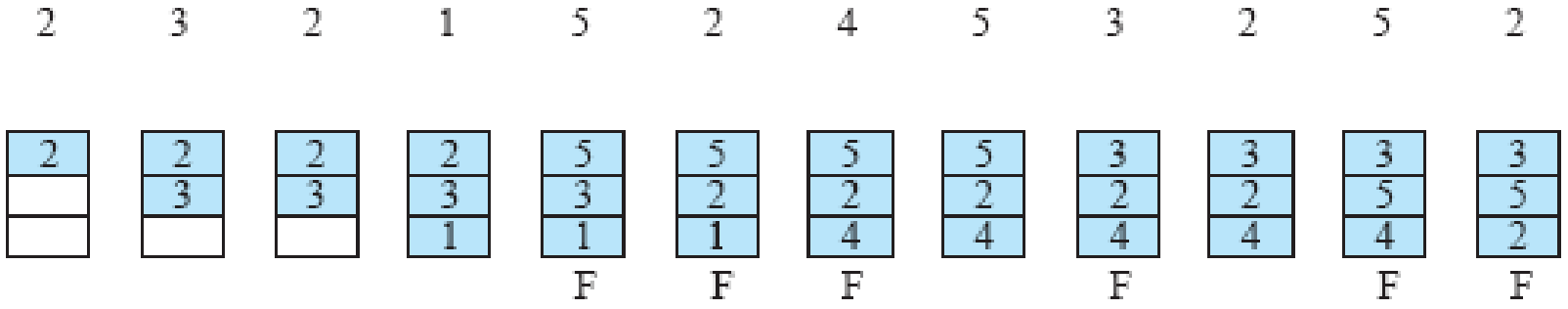
- **Total page-fault = 6**
- **Jumlah page-fault untuk penggantian page = 3**

- **Untuk slide2x selanjutnya, fokus pada page-fault untuk penggantian page untuk membandingkan page replacement algorithm**

First-In, First-Out (FIFO) Page Replacement Algorithm

- **Page frames yang lebih dahulu di-loaded ke memory yang akan lebih dahulu dikeluarkan apabila ada page-fault**
  - **Keuntungan: sederhana untuk diimplementasikan**
  - **Kekurangan: asumsi bahwa page yang paling lama berada di memory akan paling kecil peluangnya untuk diacu sering kali salah → kinerja algoritma ini kurang bagus**

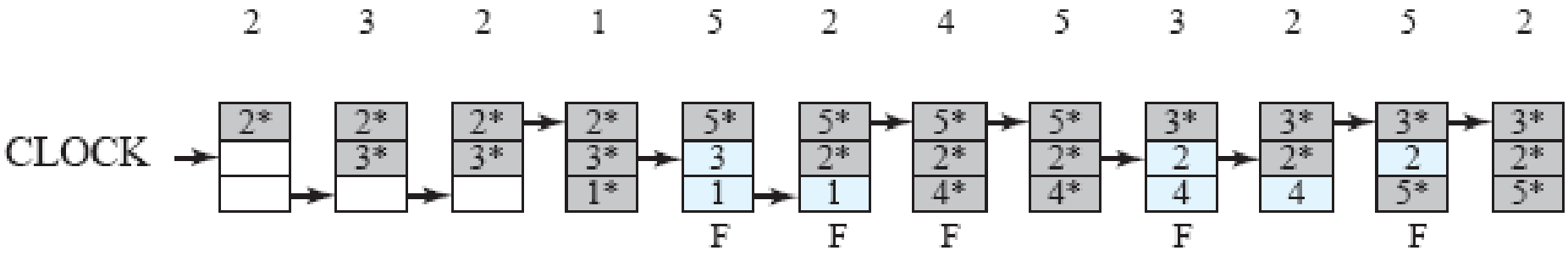
- **Contoh:**



- **Jumlah page-fault untuk penggatian page = 6**

Second Chance / Clock Page Replacement Algorithm: Contoh

- **Contoh:**



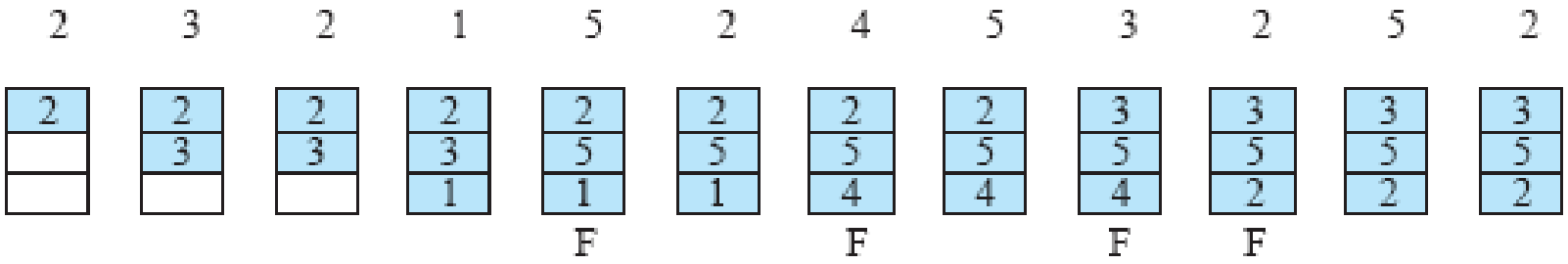
- **Bagaimana gambaran clock kalau mengacu ke slide sebelumnya?**

- **Jumlah page fault untuk penggantian page = 5**

Least-Recently Used (LRU) Page Replacement Algorithm

- **Keluarkan page yang paling lama tidak terpakai**
- **Mendekati optimal algorithm, namun implementasi lebih rumit dan lebih memakan waktu**
  - **Mengatur pengurutan page pada link-list setiap memory reference**
  - **Counter untuk tiap page yang di-update secara otomatis untuk setiap memory reference**

- **Contoh:**

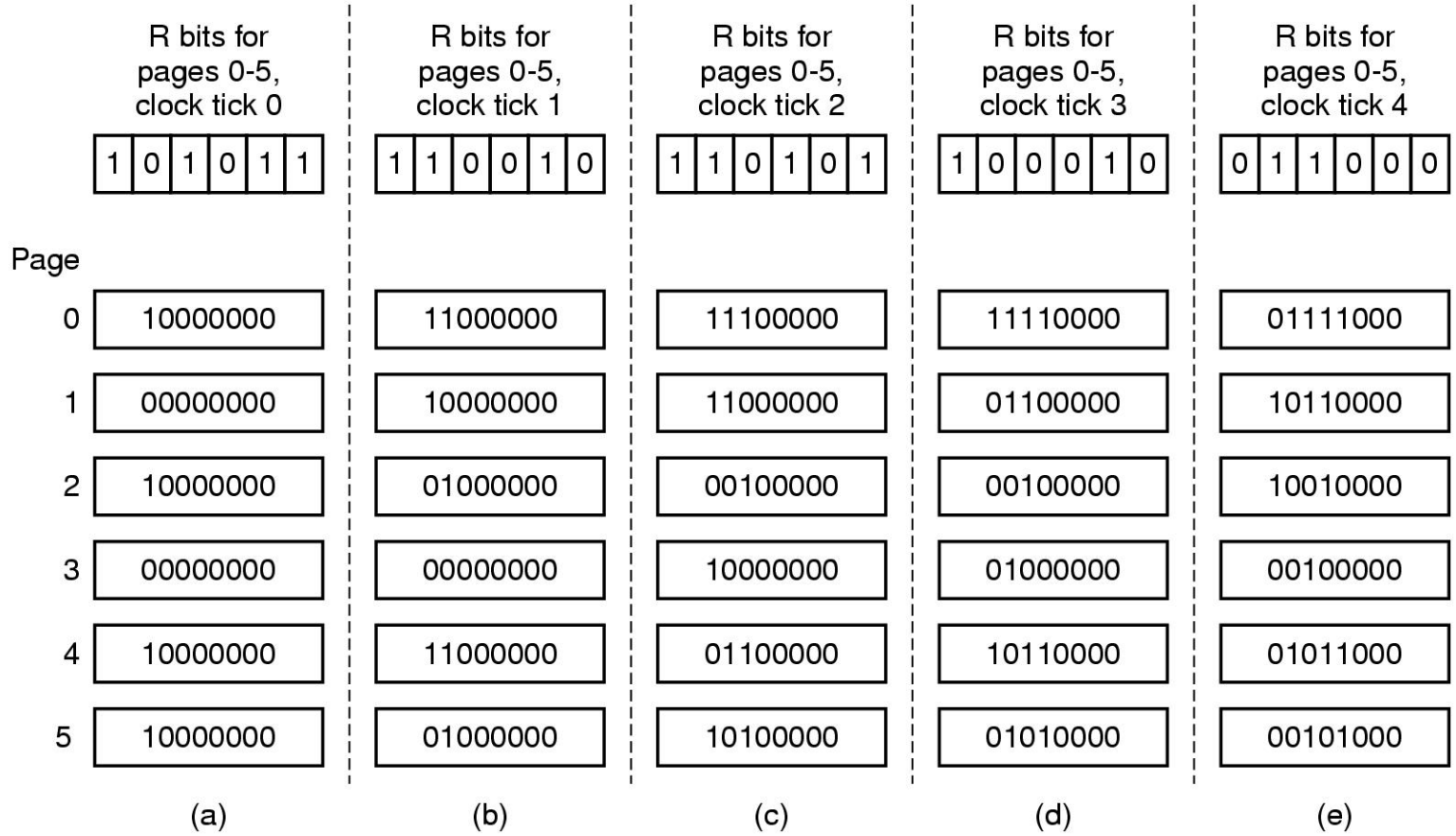


- **Jumlah page-fault untuk penggantian page = 4**

Aging Algorithm: Simulasi LRU dengan Software Counter

- **Buat counter untuk tiap Page Table Entry**
- **Secara periodik (dengan clock interrupt) periksa *R* bit:**
  - **Jika *R* = 0, nilai counter digeser 1 bit ke kanan**
  - **Jika *R* = 1, bit paling kiri pada counter = 1**
- **Nilai counter menandakan jumlah interval kapan page terakhir diakses**
- **Keluarkan page yang memiliki nilai counter terendah**

Aging Algorithm: Simulasi LRU dengan Software Counter



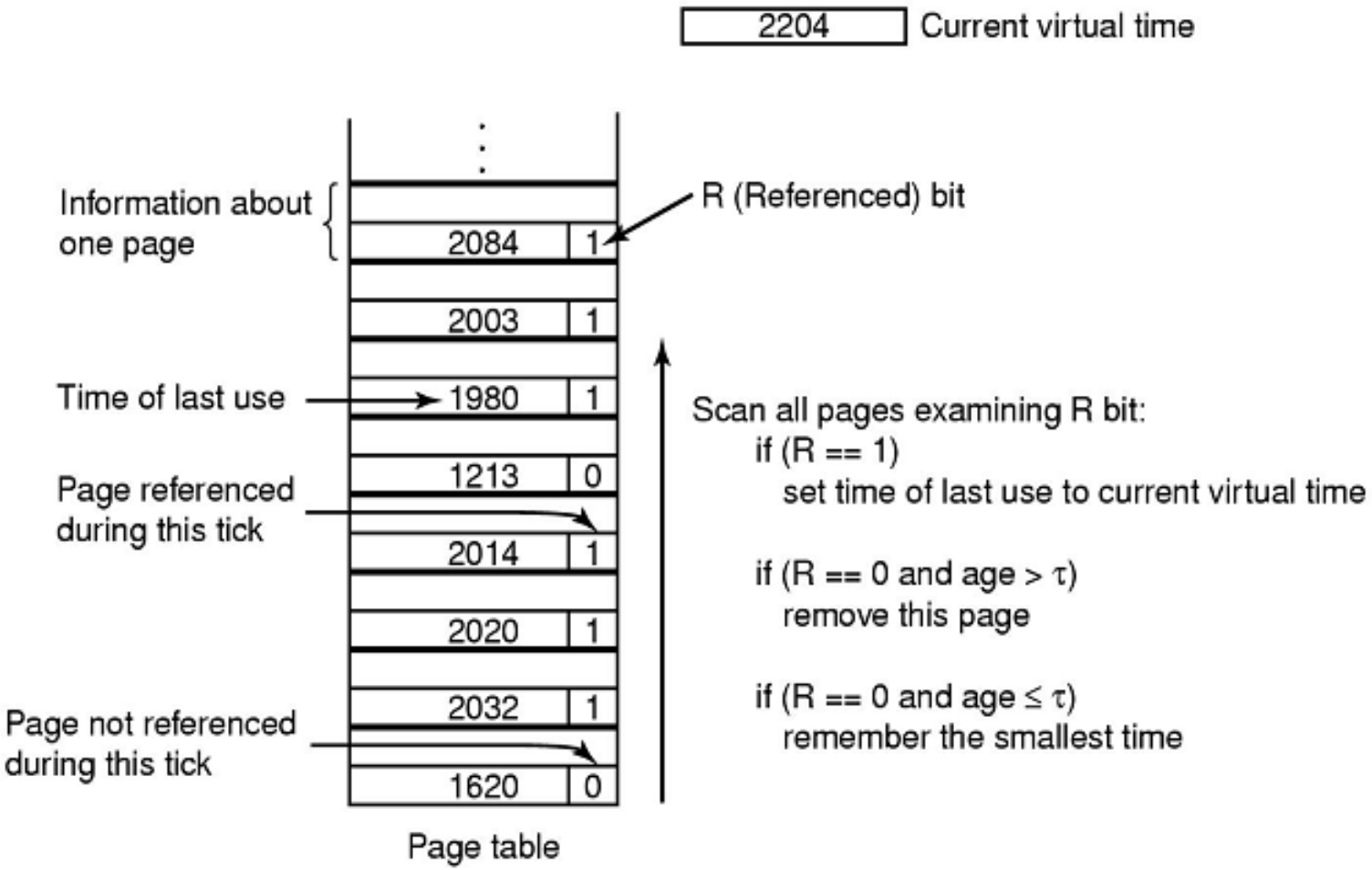
6 pages dan 5 clock ticks. Kelima clock ticks ditunjukkan oleh (a) - (e).

- **Jika ada page-fault, keluarkan page dengan nilai software counter terendah**

Working Set

- **Locality reference (Prinsip lokalitas) → observasi menunjukkan memory references cenderung tak seragam (bersifat lokal)**
  - **Temporal locality**
    - **Contoh: looping, subroutine, stack, dsb.**
  - **Spatial locality**
    - **Contoh: travesal pada array, eksekusi code secara sequential, kecenderungan programmer menempatkan variable yang terkait saling berdekatan, dsb.**
- **Konsekuensinya, program dapat berjalan secara lebih efisien saat satu himpunan bagian dari page-page berkecenderungan tinggi untuk diacu terdapat di memory**
- **Working set (himpunan kerja) adalah himpunan page-page dari suatu process yang secara aktif diacu.**
  - **Oleh karena itu, himpunan ini harus dijaga untuk berada di physical memory agar program berjalan secara efisien**

Working Set Page Replacement Algorithm: Contoh



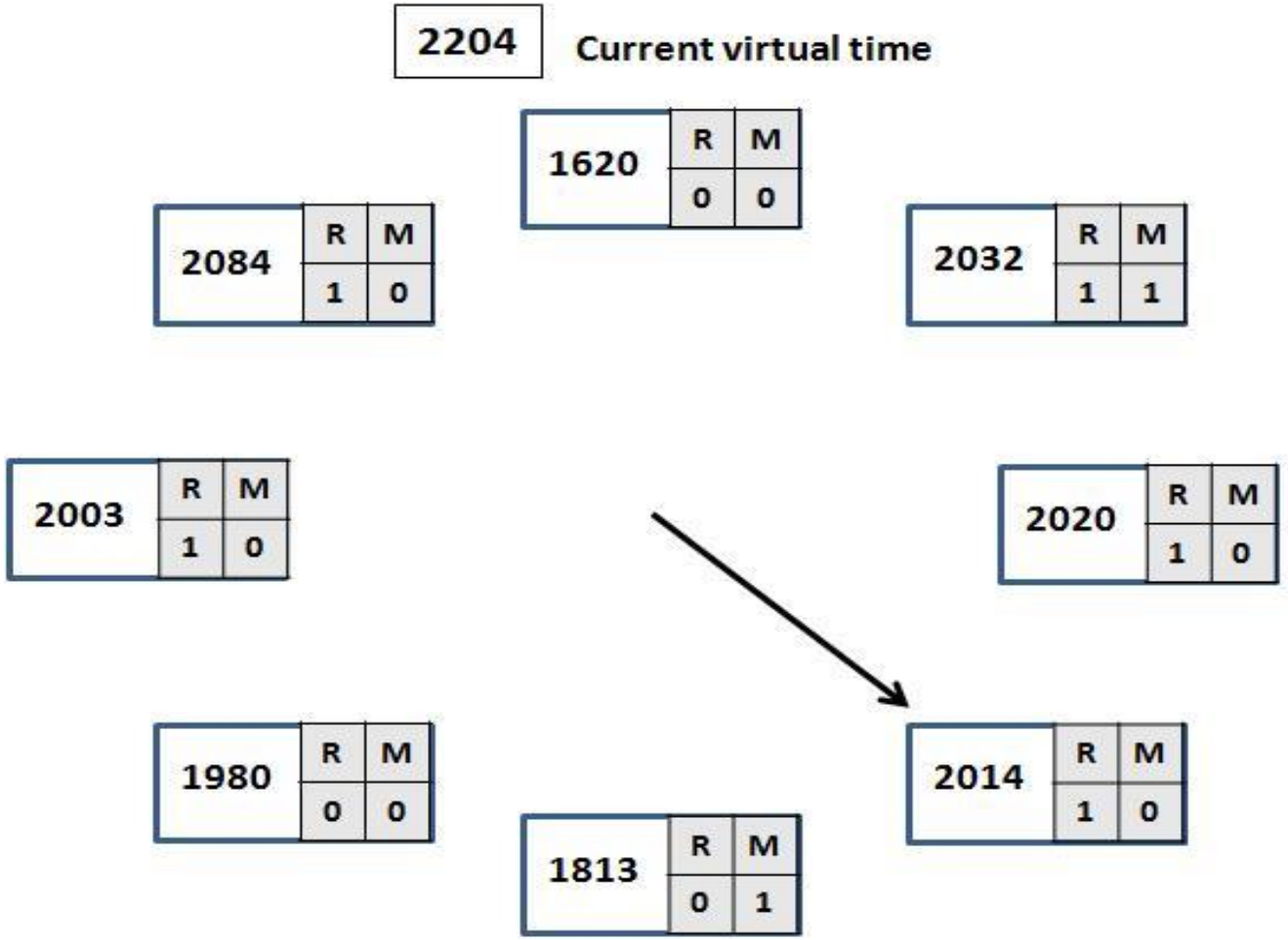
Jika window size τ=600

Tentukan page yang akan dikeluarkan

WSClock Page Replacement Algorithm

- **Struktur data: circular list of page frame (tidak perlu scan keseluruhan isi page table)**
  - **Informasi berisikan R (reference bit), M (modified: dirty/clean bit), dan time of last used (time-stamp)**
  - **R dan time-stamp di-update tiap clock tick**
- **Clock bergerak dengan arah tertentu mengitari list. Untuk tiap entry:**
  - **Jika R = 1, reset R dan update time → second chance**
  - **Jika R = 0, dan jika age < Window_Size, lanjutkan karena masih anggota working set → third chance**
  - **Jika R = 0, dan jika age > Window_Size, dan jika dirty, maka jadwalkan penulisan isi page ke disk → fourth chance**
  - **Jika R = 0, dan jika age > Window_Size, dan jika clean, maka page bisa dikeluarkan**
- **Lanjutkan ke entry berikutnya**

WSClock Page Replacement Algorithm: Contoh



Diketahui ukuran working set (window size) $\Delta = 200$.

Tentukan page-page yang akan diganti, jika terjadi page fault saat virtual time = 2204 dan 2205

Disk Scheduling Algorithms

Disk Arm Scheduling (Penjadwalan Lengan Disk)

- **Waktu yang diperlukan untuk read/write disk block:**
  - **Seek time: waktu untuk menggerakkan disk arm ke target cylinder**
  - **Rotational Delay: waktu berputarnya platter sampai target sector berada di bawah read/write head**
  - **Data transfer time**

- **Delay didominasi oleh seek time**

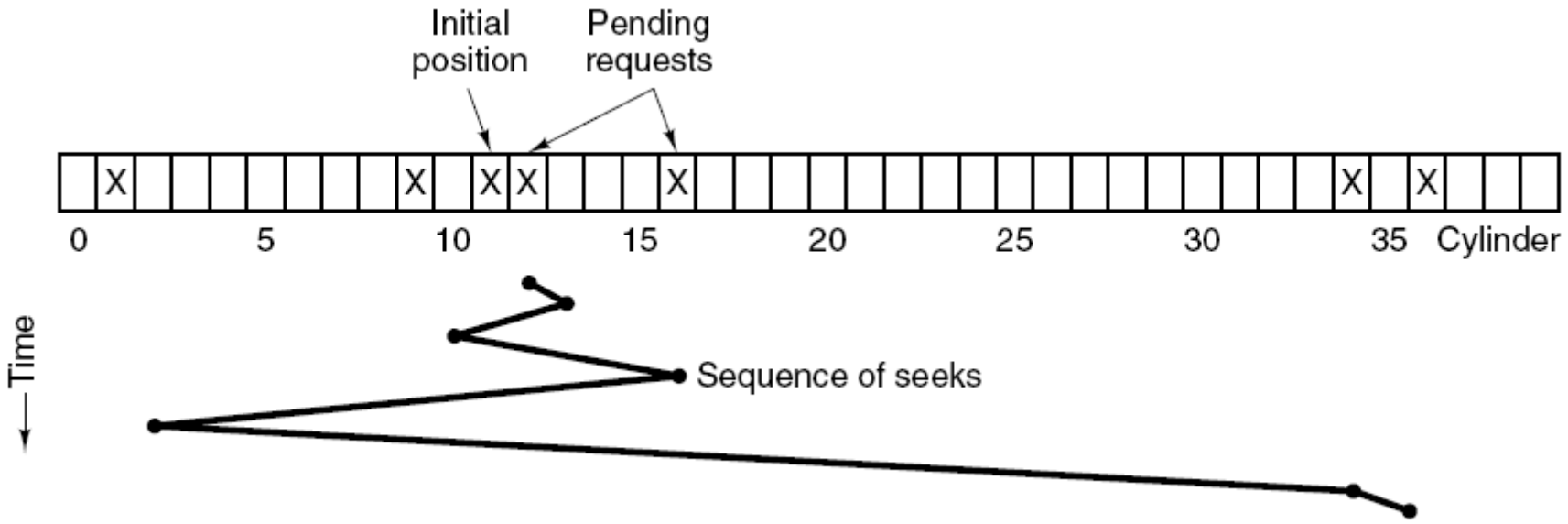
- **Disk scheduling algorithm adalah algoritma untuk meminimalkan disk seek time dan mengusahakan fairness (keadilan) atas prioritas permintaan read/write disk block**
- **Disk drives mengelola tabel permintaan read/write dengan nomor cylinder sebagai index**

First Come First Served (FCFS) Disk Scheduling Algorithm

Contoh:

- **Posisi awal = 11. Urutan permintaan: 1, 36, 16, 34, 9, 12**
- **Pergerakan: 11 → 1 → 36 → 16 → 34 → 9 → 12**
- **Total pergerakan arm = 10+35+20+18+25+3 = 111 cylinders**
- **Perlu upaya untuk mengurangi total pergerakan arm, untuk mengurangi seek time**

Shortest Seek First (SSF) Disk Scheduling Algorithm

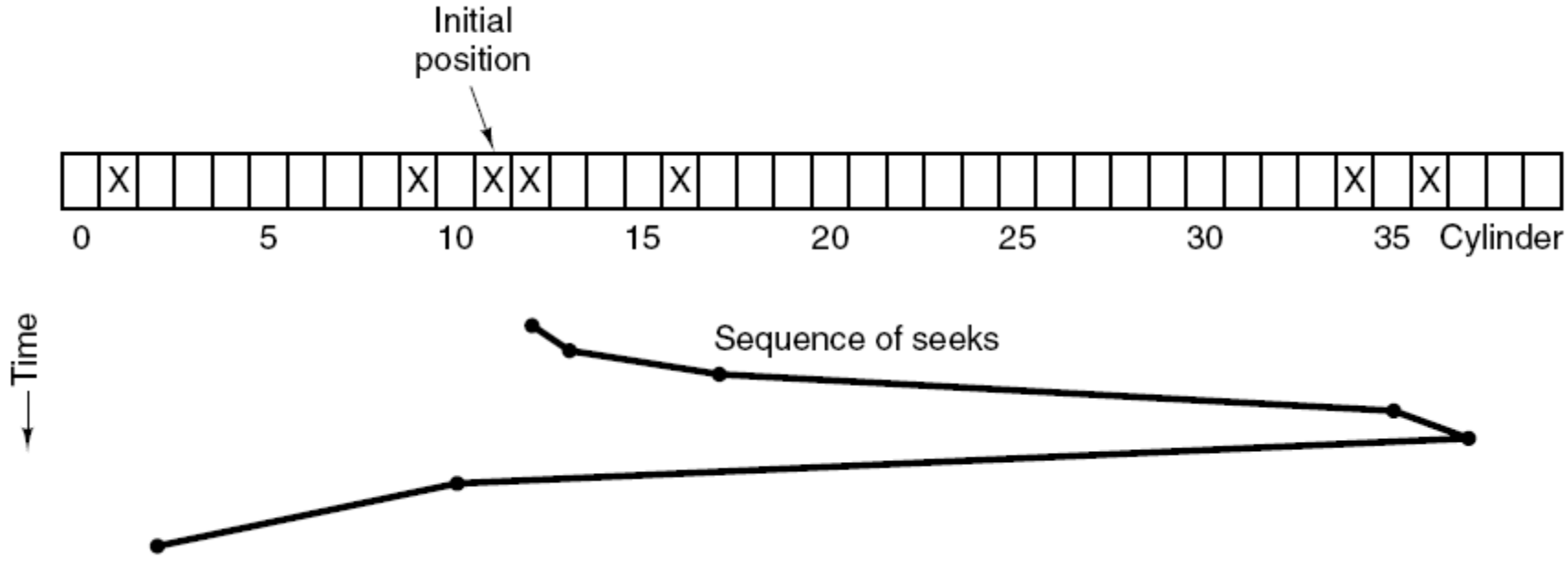


- **Kalau gunakan Shortest Seek First (SSF), total pergerakan arm = 1+3+7+15+33+2 = 61 cylinders → jauh lebih cepat dibandingkan FCFS**

- **Kekurangan: saat disk loading tinggi, ada kecenderungan disk arm untuk selalu berada di tengah sehingga tidak adil untuk permintaan block yang ada di pinggir**

Elevator atau SCAN Algorithm

- **Contoh: posisi awal = 11. Urutan permintaan: 1, 36, 16, 34, 9, 12**
- **Elevator algorithm:**
  - **Mengatasi konflik target minimum seek time dan fairness**
  - **Disk arm melanjutkan pergerakan sesuai arah (up atau down) sampai tidak ada lagi permintaan di arah tersebut, baru berbalik arah**



- **Total pergerakan arm = 1+4+18+2+27+8 = 60 cylinders**

Akhir Kuliah Minggu 14

Terima kasih atas perhatiannya!